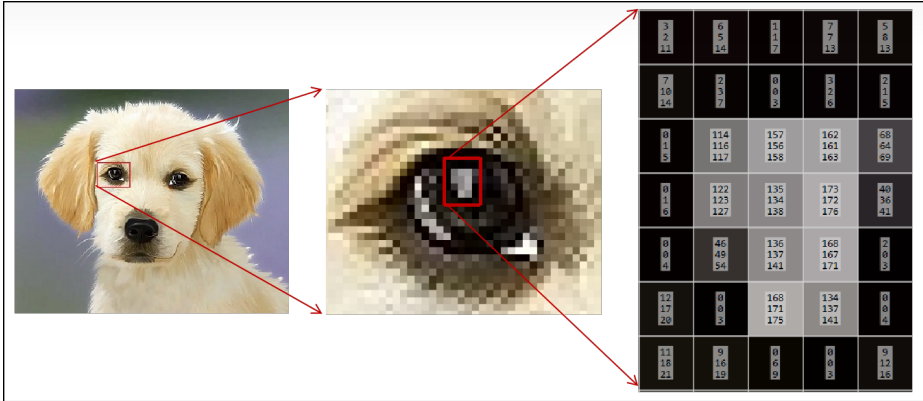




JPEG图像压缩详解和代码实现

智驱力人工智能

接地气的人工智能杂货铺

28 人赞同了该文章

收起

一、图像存储

为了有效的传输和存储图像，需要对图像数据进行压缩。依据图像的保真度，图像压缩可分为无损压缩和有损压缩。

1. 无损压缩

无损压缩的基本原理是相同的颜色信息只需保存一次。无损压缩保证解压以后的数据和原始数据完全一致，压缩时去掉或减少数据中的冗余，解压时再重新插到数据中，是一个可逆过程。无损压缩算法一般可以把普通文件的数据压缩到原来的1/2 – 1/4。

2. 有损压缩

有损压缩方式在解压后图像像素值会发生变化，解压以后的数据和原始数据不完全一致，是不可逆压缩方式。在保存图像时保留了较多的亮度信息，将冗余信息合并，合并的比例不同，压缩的比例也就不同。由于信息量减少了，所以压缩比可以很高，图像质量也会下降。

二、图像格式

常见有损的图像格式有：JPEG、WebP，常见无损的图像格式有：PNG、BMP、GIF。

通常以文件的后缀名来区分图片的格式，但有时并不准确。实际的图片格式可通过查看图片数据来确定（查看方式：Notepad++打开图片，选择“插件”->“插件管理”，安装“HEX-Editor”，安装后再次选择“插件”->“HEX-Editor”->“View in HEX”）。

以JPEG和PNG图像格式为例。JPEG格式以0xFF D8开头，以0xFF D9结尾。PNG格式以0x89 50 4E 47 0D 0A 1A 0A开头，其中50 4E 47是英文字符串“PNG”的ASCII码，以00 00 00 00 49 45 4E 44 AE 42 60 82结尾，标志着PNG数据流结束。

- 图像存储
- 无损压缩
- 有损压缩
- 图像格式
- JPEG压缩
- 色彩空间转换
- 降采样
- 离散余弦变换（DCT）
- 量化
- ZIGZAG排序
- 差分脉冲编码调制（DPC...
- DC系数中间格式
- 行程长度编码（RLC）对...
- AC系数中间格式
- 熵编码

图像格式

文件头部

文件尾部

JPG

hex:jpg

Address	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00000000	ff	d8	ff	e0	00	10	4a	46	49	46	00	01	01	01	00	78
00000010	00	78	00	00	ff	e1	00	5a	45	78	69	66	00	00	4d	4d
00000020	00	2a	00	00	00	08	00	05	03	01	00	05	00	00	00	01
00000030	00	00	00	4a	03	03	00	01	00	00	00	01	00	00	00	00
00000040	51	10	00	01	00	00	00	01	01	00	00	00	51	11	00	04
00000050	00	00	00	01	00	00	12	74	51	12	00	04	00	00	00	01
00000060	00	00	12	74	00	00	00	00	00	86	a0	00	00	b1	8f	
00000070	ff	db	00	43	00	02	01	01	02	01	01	02	02	02	02	02
00000080	02	02	02	03	05	03	03	03	03	06	04	04	03	05	07	
00000090	06	07	07	07	06	07	07	08	09	0b	09	08	08	0a	08	07
000000a0	07	0a	0d	0a	0a	0b	0c	0c	0c	07	09	0e	0f	0d	0c	
000000b0	0e	0b	0c	0c	0c	ff	db	00	43	01	02	02	02	03	03	03
000000c0	06	03	03	06	0c	08	07	08	0c	0c	0c	0c	0c	0c	0c	0c
000000d0	0c	0c	0c	0c	0c	0c	0c	0c	0c	0c	0c	0c	0c	0c	0c	0c

hex:jpg

Address	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00006e90	59	62	56	56	1e	8c	7f	91	a2	8a	e5	a7	26	cd	d4	15
00006ea0	ec	46	7f	d2	60	55	ee	bd	98	8e	4f	b5	66	de	c0	37
00006eb0	95	91	59	97	76	d5	38	c9	f7	1f	fd	6a	28	a3	95	35
00006ec0	a9	aa	49	68	8c	db	eb	54	82	56	44	6d	cb	82	14	81
00006ed0	8c	10	70	78	f4	fa	56	6b	31	b0	90	c2	c8	ad	03	8c
00006ee0	85	23	85	f6	1e	de	dd	a8	a2	b0	ad	4d	2d	83	ed	23
00006ef0	3f	52	0b	a6	41	33	92	5a	15	c1	3c	64	af	bf	f9	eb
00006f00	59	2e	cb	04	fb	51	d5	a3	93	a1	5e	8d	e8	47	f9	f6
00006f10	a2	8a	f3	ab	f6	3a	69	ee	3a	37	f2	9b	6f	71	5b	9a
00006f20	3e	b2	20	b8	8d	a3	5f	2e	4c	0c	a9	19	56	fe	5d	7f
00006f30	9d	14	56	78	79	34	d1	15	25	7d	4e	fb	4d	b8	fb	4c
00006f40	51	b3	2e	de	3e	52	07	6e	b8	3c	fa	1d	2a	1d	62	
00006f50	47	80	6d	99	56	64	61	8f	97	ef	63	3e	9d	3f	fa	fd
00006f60	c8	c5	14	57	d5	52	fe	09	e3	4b	f8	a7	ff	d9		

PNG

hex:png

Address	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00000000	89	50	4e	47	0d	0a	1a	0a	00	00	00	0d	49	48	44	52
00000010	00	00	01	40	00	00	01	08	08	06	00	00	00	2e	9b	2d
00000020	ab	00	00	00	01	73	52	47	42	00	ae	ce	1c	e9	00	00
00000030	00	04	67	41	4d	41	00	00	b1	8f	0b	fc	61	05	00	00
00000040	00	09	70	48	59	73	00	00	12	74	00	00	12	74	01	de
00000050	66	1f	78	00	00	ff	a5	49	44	41	54	78	5e	bc	fd	79
00000060	b8	77	c9	75	d7	87	d6	99	e7	e1	9d	87	9e	e7	49	6a
00000070	a9	07	b5	c6	d6	3c	59	b2	25	4f	18	6c	83	6c	61	1b
00000080	30	8e	9f	24	b6	b9	24	0c	41	17	42	72	93	4b	c8	05
00000090	ee	05	02	09	89	81	60	c0	0e	60	83	9f	38	0e	b6	c1
000000a0	d8	c6	b2	2d	c9	96	35	58	53	b7	7a	7a	fb	9d	df	f7
000000b0	cc	f3	39	f7	f3	f9	ae	bd	cf	39	6f	0f	e6	3e	f9	e3
000000c0	d6	ef	ec	b3	f7	ae	5d	b5	6a	d5	aa	b5	56	ad	55	55
000000d0	bb	f6	c0	f7	7f	fc	bf	d8	6b	af	12	f6	f6	fa	e0	
000000e0	d0	60	5d	ef	ec	b6	9d	81	d6	06	b8	f7	3c	b8	b3	d7
000000f0	76	f8	91	24	e9	06	b8	ef	03	bb	dc	70	0c	ee	b4	dd

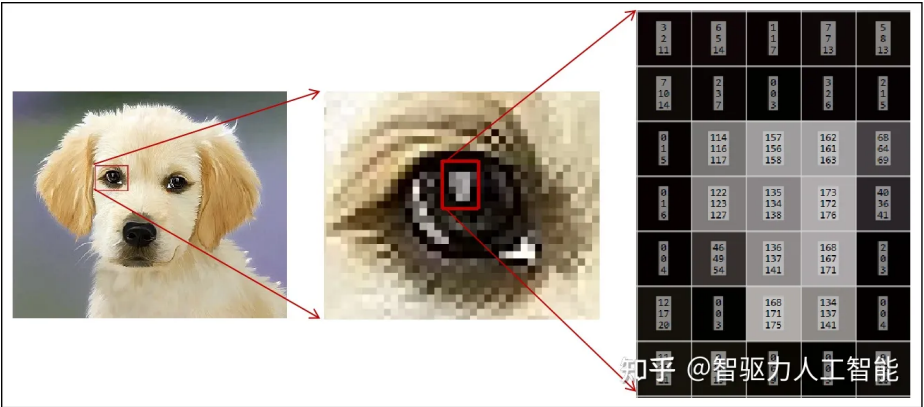
hex:png

Address	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00027780	bb	f7	94	31	49	bb	ea	18	14	61	06	6f	a2	7a	d8	67
00027790	aa	1f	0a	a2	bf	e3	fc	03	e1	81	c4	56	fe	3d	aa	53
000277a0	59	f5	ea	e6	e0	7e	76	7a	5a	fc	21	8d	83	ef	7a	e2
000277b0	ef	f2	11	2c	ee	91	d4	0a	2c	ee	bb	20	8d	8f	49	eb
000277c0	76	52	a3	e1	f8	f8	38	c6	ff	8a	6b	67	59	ad	c4	ae
000277d0	db	73	68	c5	b4	e5	3d	61	64	81	6b	43	96	ae	2f	3c
000277e0	09	3e	f2	96	73	7b	1f	79	77	9c	02	4d	8b	3a	5a	8f
000277f0	31	16	76	1d	a7	e9	46	8b	a6	86	ce	09	02	bb	8c	36
00027800	3e	69	1c	09	31	32	34	4a	8e	45	45	93	e1	7b	83	9f
00027810	b1	c6	40	37	ec	d3	52	fa	20	84	6a	b8	2a	d5	e9	c9
00027820	29	b2	ad	b5	7f	86	32	95	69	bf	03	e7	92	23	6d	7a
00027830	8e	1b	74	d7	36	dd	09	4d	6d	a0	5c	87	88	0c	82	44
00027840	fa	75	e2	3f	47	ae	a2	5b	09	26	e3	5e	9c	0a	74	5c
00027850	dc	b4	f9	3f	46	8f	8f	8f	8f	8f	8f	8f	8f	8f	8f	8f
00027860	f8	37	1d	2f	ef	50	fe	43	43	43	43	43	43	43	43	43
00027870	4e	44	ae	42	60	82										

三、JPEG压缩

上文的图例是图像文件实际保存的数据，也就是图像压缩后的数据。本文以JPEG格式为例讲解图像压缩的过程。JPEG的文件格式一般有两种文件扩展名：.jpg和.jpeg，这两种扩展名的实质是相同的，我们可以把.jpg的文件改名为.jpeg，而对文件本身不会有任何影响。严格来讲，JPEG的文件扩展名应该为.jpeg，由于DOS时代的8.3文件名命名原则，就使用了.jpg的扩展名。

下文以小狗图像为例，详述图片压缩具体过程，图像分辨率是320x264。首先看下图：



通常我们看到的彩色图像是三通道或四通道图像。三通道图像是指有RGB三个通道，R：红色，G：绿色，B：蓝色。四通道图像是在三通道的基础上加了Alpha通道，Alpha通道用来衡量一个像素的透明度。当Alpha为0时，该像素完全透明；当Alpha为255时，该像素完全不透明。四通道图像只有PNG格式支持。

图中小狗是三通道图像，有320x264个像素点，每个像素点由三个值表示，如上图右侧小狗眼睛部分，黑色区域每个通道的像素值较小如（3，2，11），白点部分像素值较高如（114，116，117）。图中共84480个像素，每个像素用24位表示，若直接存储需要占用84480*24/1024=247.5KB，为了有效地传输和存储图像，有必要对图像做压缩。JPEG压缩步骤如下。

1. 色彩空间转换

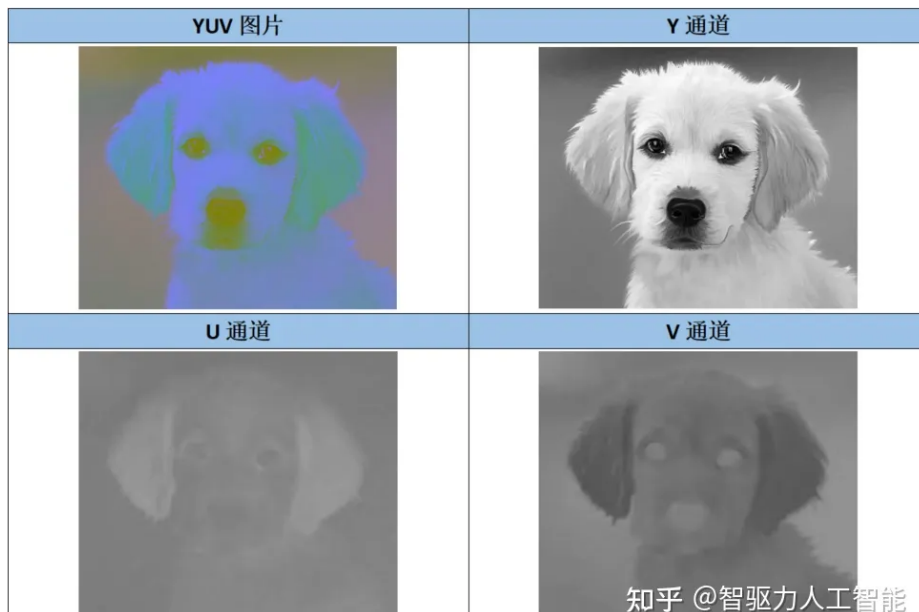
JPEG采用YUV颜色空间，“Y”表示明亮度，也就是灰度值；“U”和“V”表示色度，用于描述图像色彩和饱和度。因为人眼对亮度比较敏感，而对于色度不那么敏感，可以在UV维度大量缩减信息，所以先将RGB的数据转换到YUV色彩空间。转换公式：

- $Y = 0.299R + 0.587G + 0.114B$
- $U = 0.5R - 0.4187G - 0.0813G + 128$
- $V = -0.1687R - 0.3313G + 0.5B + 128$

python 实现

```
import cv2
import numpy as np
# opencv 读取的图片是BGR顺序
image = cv2.imread('data/dog.jpg')
h, w, c = image.shape
# 色彩空间转换 BGR -> YUV
image_yuv = np.zeros_like(image, dtype=np.uint8)
for line in range(h):
    for row in range(w):
        B = image[line, row, 0]
        G = image[line, row, 1]
        R = image[line, row, 2]
        Y = np.round(0.299*R + 0.587*G + 0.114*B)
        U = np.round(0.5*R - 0.4187*G - 0.0813*B + 128)
        V = np.round(-0.1687*R - 0.3313*G + 0.5*B + 128)
        image_yuv[line, row, :] = (Y, U, V)
# 保存图像
cv2.imwrite('Y.png', image_yuv[:, :, 0])
cv2.imwrite('U.png', image_yuv[:, :, 1])
cv2.imwrite('V.png', image_yuv[:, :, 2])
cv2.imwrite('YUV.png', image_yuv)
```

结果展示



2. 降采样

由于人眼对色度不敏感，直接将U、V分量进行色度采样，JPEG压缩算法采用YUV 4:2:0的色度抽样方法。4:2:0表示对于每行扫描的像素，只有一种色度分量以2:1的抽样率存储，也就是说每隔一行/列取值，偶数行取U值，奇数行取V值，UV通道宽度和高度分别降低为原来的1/2。



python 实现

```
# 色彩空间转换 BGR -> YUV 4:2:0
def RGB2YUV420(image):
    h, w, c = image.shape
    image_y = np.zeros((h, w), dtype=np.uint8)
    image_u = np.zeros(((h-1)//2+1, (w-1)//2+1), dtype=np.uint8)
    image_v = np.zeros(((h-1)//2+1, (w-1)//2+1), dtype=np.uint8)
    for line in range(h):
        for row in range(w):
            B = image[line, row, 0]
            G = image[line, row, 1]
            R = image[line, row, 2]
            Y = np.round(0.299*R + 0.587*G + 0.114*B)
            image_y[line, row] = Y
            if line % 2 == 0 and row % 2 == 0:
                U = np.round(0.5*R - 0.4187*G - 0.0813*B + 128)
                image_u[line//2, row//2] = U
            if line % 2 == 1 or row % 2 == 1:
                V = np.round(-0.1687*R - 0.3313*G + 0.5*B + 128)
                image_v[line//2, row//2] = V
    return image_y, image_u, image_v
```

结果展示



3. 离散余弦变换 (DCT)

人类视觉对高频信息不敏感，利用离散余弦变换可分析出图像中高低频信息含量，进而压缩数据。

JPEG中将图像分为8*8的像素块，对每个像素块利用离散余弦变换进行频域编码，生成一个新的8*8的数字矩阵。对于不能被8整除的图像大小，需对图像填充使其可被8整除，通常使用0填充。由于离散余弦变换需要定义域对称，所以先将矩阵中的数值左移128，使值域范围在[-128, 127]。

二维离散余弦变换公式为：

$$F(x, y) = \alpha(x) * \alpha(y) * \sum_{i=0}^7 \sum_{j=0}^7 f(i, j) \cos\left(\frac{2i+1}{16} x \pi\right) \cos\left(\frac{2j+1}{16} y \pi\right) \quad x, y = 0, 1, \dots, 7$$

$$\text{其中, } \alpha(u) = \begin{cases} 1/\sqrt{8} & u = 0 \\ 1/2 & u \neq 0 \end{cases}$$

python 实现

```
import math
def alpha(u):
    if u==0:
        return 1/np.sqrt(8)
    else:
        return 1/2

def block_fill(block):
    block_size = 8
    dst = np.zeros((block_size, block_size), dtype=np.uint8)
    h, w = block.shape
    dst[:h, :w] = block
    return dst

def DCT_block(img):
    block_size = 8
    img = block_fill(img)
    img_fp32 = img.astype(np.float32)
    img_fp32 -= 128
    img_dct = np.zeros((block_size, block_size), dtype=np.float32)
    for line in range(block_size):
        for row in range(block_size):
            n = 0
            for x in range(block_size):
                for y in range(block_size):
                    n += img_fp32[x,y]*math.cos(line*np.pi*(2*x+1)/16)*math.cos(r
            img_dct[line, row] = alpha(line)*alpha(row)*n
    return np.ceil(img_dct)

def DCT(image):
    block_size = 8
    h, w = image.shape
    dlist = []
    for i in range((h + block_size - 1) // block_size):
        for j in range((w + block_size - 1) // block_size):
            img_block = image[i*block_size:(i+1)*block_size, j*block_size:(j+1)*t
            # 处理一个像素块
            img_dct = DCT_block(img_block)
            dlist.append(img_dct)
    return dlist

img_dct = DCT(image_y)
```

结果展示

图像像素	值域转换（减 128）	离散余弦变换
[[203 191 195 211 191 174 203 196] [207 198 202 211 194 177 203 197] [211 211 210 209 197 182 203 198] [212 214 216 208 198 184 201 197] [213 215 216 209 198 182 202 208] [217 218 218 209 198 188 208 212] [221 222 222 211 202 196 214 216] [223 224 226 217 210 205 217 217]]	[[75. 63. 67. 83. 63. 46. 75. 68.] [79. 70. 74. 83. 66. 49. 75. 69.] [83. 83. 82. 81. 69. 54. 75. 70.] [84. 86. 88. 80. 70. 56. 73. 69.] [85. 87. 88. 81. 70. 54. 74. 80.] [89. 90. 90. 81. 70. 60. 80. 84.] [93. 94. 94. 83. 74. 68. 86. 88.] [95. 96. 98. 89. 82. 77. 89. 89.]]	[[621. 43. 23. -38. 11. 21. -19. 14.] [-52. -2. -9. 2. 13. 14. -8. -0.] [6. -12. -1. 6. 5. 8. -0. -2.] [-12. -4. 5. -4. 6. 3. -1. -0.] [-0. -0. -4. 1. 1. -0. 1. 1.] [-0. 2. 1. 2. -3. 1. -0. 1.] [0. 0. 0. 0. 0. 0. 0. 0.] [0. 0. 0. 0. 0. 0. 0. 0.]

4. 量化

每个8*8的像素块经离散余弦变换后生成一个8*8的浮点数矩阵，量化的过程则是去除矩阵中的高频信息，保留低频信息。JPEG算法提供了两张标准化系数矩阵，分别处理亮度数据和色差数据，表示 50% 的图像质量。

$$Q_Y = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix} \quad Q_C = \begin{bmatrix} 17 & 18 & 24 & 47 & 99 & 99 & 99 & 99 \\ 18 & 21 & 26 & 66 & 99 & 99 & 99 & 99 \\ 24 & 26 & 56 & 99 & 99 & 99 & 99 & 99 \\ 47 & 66 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \\ 99 & 99 & 99 & 99 & 99 & 99 & 99 & 99 \end{bmatrix}$$

量化的过程：使用DCT变换后的浮点矩阵除以量化表中数值，然后取整。量化表是控制JPEG压缩比的关键，可以根据输出图片的质量来自定义量化表，通常自定义量化表与标准量化表呈比例关系，表中数字越大则质量越低，压缩率越高。

python 实现

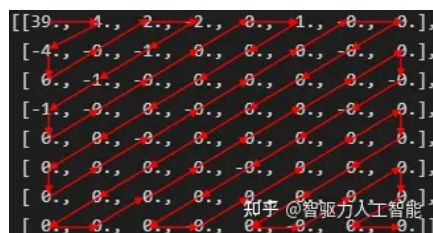
```
def quantization(blocks, Q):
    img_quan = []
    for block in blocks:
        img_quan.append(np.round(np.divide(block, Q)))
    return img_quan
img_quan = quantization(img_dct, Qy)
```

结果展示

输入数据	量化表	量化结果
[[621. 43. 23. -38. 11. 21. -19. 14.] [-52. -2. -9. 2. 13. 14. -8. -0.] [6. -12. -1. 6. 5. 8. -0. -2.] [-12. -4. 5. -4. 6. 3. -1. -0.] [-0. -0. -4. 1. 1. -0. 1. 1.] [-0. 2. 1. 2. -3. 1. -0. 1.] [2. 1. 1. 1. -0. -0. -0. -0.] [1. -0. 2. -1. 2. -1. -0. -0.]]	Q_Y	[[39., 4., 2., -2., 0., 1., -0., 0.] [-4., -0., -1., 0., 0., 0., -0., 0.] [0., -1., -0., 0., 0., 0., 0., -0.] [-1., -0., 0., -0., 0., 0., 0., 0.] [0., 0., -0., 0., 0., 0., 0., 0.] [0., 0., 0., -0., 0., 0., 0., 0.] [0., 0., 0., 0., -0., 0., 0., 0.] [0., 0., 0., 0., 0., 0., 0., 0.]

5. ZIGZAG排序

排序规则如图：



python 实现

```
def zigzag(blocks):
    block_list = []
    for block in blocks:
        zlist = []
        w, h = block.shape
        if w != h:
            return None
        max_sum = w + h - 2
        for _s in range(max_sum + 1):
            if _s % 2 == 0:
                for i in range(_s, -1, -1):
                    j = _s - i
                    if i >= w or j >= h:
                        continue
                    zlist.append(block[i,j])
            else:
                for j in range(_s, -1, -1):
                    i = _s - j
                    if i >= w or j >= h:
                        continue
                    zlist.append(block[i,j])
        block_list.append(zlist)
    return block_list

zglist = zigzag(img_quan)
```

结果展示

[39.0, 4.0, -4.0, 0.0, -0.0, 2.0, -2.0, -1.0, -1.0, -1.0, 0.0, -0.0, -0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.0, -0.0, 0.0, 0.0, -0.0, 0.0, -0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.0, -0.0, 0.0, -0.0, 0.0, 0.0, -0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

6. 差分脉冲编码调制 (DPCM) 对直流系数 (DC) 编码

对像素矩阵做DCT变换，相当于将矩阵的能量压缩到第一个元素中，左上角第一个元素被称为直流（DC）系数，其余的元素被称为交流（AC）系数。JPEG将量化后的频域矩阵中的DC系数和AC系数分开编码。使用DPCM技术，对相邻图像块量化DC系数的差值进行编码；使用行程长度编码（RLE）对AC系数编码。需要注意的一点是，对AC系数的RLE编码是在8x8的块内部进行的，而对DC系数的DPCM编码是在整个图像上若干个8x8的块之间进行的。

差值编码原理：样值与前一个（相邻）样值的差值，则这些差值大多数是很小的或为零，可以用短码来表示；而对于出现几率较差的差值，用长码表示，这样可以使总体码数下降；采用对相邻样值差值进行变字节长编码的方式称为差值编码，又称为差分脉码调制（DPCM）。

8x8的图像块经过DCT变换后，得到的直流系数特点：

- 系数值较大;
- 相邻图像块的系数值变换不大。

python 实现

```
def DPCM(zglist):
    res_dpcm = []
    for i in range(len(zglist)):
        if i == 0:
            res_dpcm.append(zglist[i][0])
            continue
        res_dpcm.append(zglist[i][0]-zglist[i-1][0])
    return res_dpcm
```

```
res_dpcm = DPCM(zglist)
```

结果展示

[50.0, -2.0, -13.0, -7.0, -3.0, 0.0, -1.0, 0.0, -1.0, -2.0, -0.0, -1.0, 0.0, -1.0, -0.0, -1.0, 0.0, -0.0, -0.0, -0.0, -0.0, 0.0, -0.0, -0.0, ..., -0.0, 0.0, -0.0, 0.0, -0.0]

7. DC系数中间格式

JPEG中为了更进一步节约空间，不直接保存数据的具体数值，而是将数据按照位数分为16组，保存在表里面。这也就是所谓的变长整数编码VLI。编码VLI表如下：

数值	组	实际保存值
0	0	-
-1, 1	1	0, 1
-3, -2, 2, 3	2	00, 01, 10, 11
-7, -6, -5, -4, 4, 5, 6, 7	3	000, 001, 010, 011, 100, 101, 110, 111
-15, -14, ..., -8, 8, ..., 14, 15	4	0000, 0001, ..., 0111, 1000, ..., 1110, 1111
-31, ..., -16, 16, ..., 31	5	00000, ..., 01111, 10000, ..., 11111
-63, ..., -32, 32, ..., 63	6	000000, ..., 011111, 100000, ..., 111111
...		
-32767, ..., -16384, 16384, 32767	15	0000000000000000, ..., 0111111111111111, 1000000000000000, ..., 1111111111111111

以第一个block和第二个block为例，DPCM结果是50，通过查找VLI编码表该值位于VLI表格的第6组，因此可以写成(6)(50)的形式，即为DC系数的中间格式。

8. 行程长度编码（RLC）对交流系数（AC）编码

具有相同颜色并且是连续的像素数目称为行程长度。RLC编码简单直观，编码/解码速度快。例如,字符串AAABCDDBBBBBB利用RLE原理可以压缩为3ABC8D5B。在JPEG编码中，使用的数据对是（两个非零AC系数之间连续0的个数，下一个非零AC系数的值）。注意，如果AC系数之间连续0的个数超过16，则用一个扩展字节(15,0)来表示16连续的0。

python 实现

```
def rlc(zglist):
    res_ac = []
    for i in range(len(zglist)):
        ac = []
        zg = zglist[i]
        zero_num = 0
        for k in range(1, len(zg)):
            if zg[k] != 0:
                ac.append((zero_num, zg[k]))
                zero_num = 0
            else:
                zero_num += 1
        if zero_num:
            ac.append((0, 0))
        res_ac.append(ac)
    return res_ac
res_ac = rlc(zglist)
```


结果展示

zigzag结果: [50.0, -2.0, -13.0, -7.0, -3.0, 0.0, -1.0, 0.0, -1.0, -2.0, -0.0, -1.0, 0.0, -1.0, -0.0, -1.0, 0.0, -0.0, -0.0, -0.0, -0.0, 0.0, -0.0, -0.0, ..., -0.0, 0.0, -0.0, 0.0, -0.0]

RLC编码结果: [(0, -2.0), (0, -13.0), (0, -7.0), (0, -3.0), (1, -1.0), (1, -1.0), (0, -2.0), (1, -1.0), (1, -1.0), (1, -1.0), (0, 0)]

9. AC系数中间格式

RLC编码结果: [(0, -2.0), (0, -13.0), (0, -7.0), (0, -3.0), (1, -1.0), (1, -1.0), (0, -2.0), (1, -1.0), (1, -1.0), (1, -1.0), (0, 0)]

对每组数据第二个数进行VLI编码, (0, -2.0)第二个数是-2.0, 查找VLI编码表是第2组, 所以可将其写(0, 2), -2.0。同理, AC系数中间格式可写成以下形式:

(0, 2), -2.0, (0, 4), -13.0, (0, 3), -7.0, (0, 2), -3.0, (1, 1), -1.0, (1, 1), -1.0, (0, 2), -2.0, (1, 1), -1.0, (1, 1), -1.0, (1, 1), -1.0, (0, 0)

10. 熵编码

JPEG基本系统规定采用Huffman编码。Huffman编码时DC系数与AC系数分别采用不同的Huffman编码表, 对于亮度和色度也采用不同的Huffman编码表。因此, 需要4张Huffman编码表才能完成熵编码的工作。具体的Huffman编码采用查表的方式来高效地完成。

上文中8x8像素块的中间格式:

- DC: (6)(50), 数字6查DC亮度Huffman编码表是1110, 数字50查VLI编码表是110010。
- AC: (0, 2), -2.0, (0, 4), -13.0, (0, 3), -7.0, (0, 2), -3.0, (1, 1), -1.0, (1, 1), -1.0, (0, 2), -2.0, (1, 1), -1.0, (1, 1), -1.0, (1, 1), -1.0, (0, 0), (0, 2) 查AC亮度Huffman编码表是01, -2.0查VLI编码表是01。

因此, 这个8x8的亮度像素块信息压缩后的数据流为

1110110010,0101,10110010,100000,0100,11000,11000,0101,11000,11000,11000,1010。总共65比特, 压缩比为 $(64 \times 8 - 65) / (64 \times 8) \times 100\% = 87.3\%$

以上是JPEG压缩的整个过程, 最终将所有编码结果整合并按JPEG规范格式存储, 即可得到jpg格式的图像文件。



知乎 @智驱力人工智能

智驱力-科技驱动生产力
www.aidrive-tech.com

编辑于 2023-01-29 13:55 · IP 属地山东

压缩

JPEG

Python

写下你的评论...

4 条评论

默认

最新



可为

文章很棒 一目了然👍

10-08 · IP 属地海南

● 回复 ● 喜欢



一只小白

RLC编码为什么是对DPCM编码结束后的矩阵进行呢？

05-16 · IP 属地北京

● 回复 ● 喜欢



上班耽误我赚钱

太专业了，一张图片这么复杂。图片格式对应一种压缩算法吗？

05-16 · IP 属地河北

● 回复 ● 喜欢



小太阳

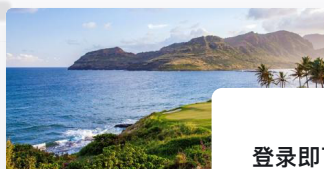
条理清晰，很有帮助，感谢分享



04-10 · IP 属地捷克

● 回复 ● 喜欢

推荐阅读



前端实现图片压缩及

尹成诺

发表于

图像压缩——图像基础知识

因为毕业论文跟图像压缩有关，所

登录即可查看 超5亿 专业优质内容

超 5 千万创作者的优质提问、专业回答、深度文章和精彩视频尽在知乎。

立即登录/注册

▲ 赞同 28 ▼

● 4 条评论

🔗 分享

❤ 喜欢

